NUEVO PIM · CATALOGACIÓN POR API

Catalogar un producto, paso por paso

El flujo end-to-end entre el seller, el PIM y los canales.

Explicado para el equipo funcional y para quien consumirá la API.

Propuesta 16-oct-2026 | Alineación con Mercado Libre | Ejemplos ilustrativos

Contrato no ratificado · nombres y JSON son referencia, no decisión final

LIVERPOOL · MERCADERÍAS + MARKETPLACE

01 · REENCUADRE

¿Cuántas APIs necesita el seller para catalogar?

Al inventario le salieron 45 rutas. Al revisarlas por el flujo real quedan 41, y solo 8 catalogan.

41

RUTAS EN EL INVENTARIO INICIAL

—

SE FUNDEN

−4 rutas

el dato ya viene en otra respuesta

8 **llamadas para catalogar**

2 se hacen una sola vez (quién soy y dónde recibo avisos).
Las otras 6 son por producto.

33 **rutas del ciclo que sigue**

Lotes, correcciones, variantes, imágenes, historial, canales y modificaciones con permiso.

8 + 33 = 41 rutas bajo /partner/v1 · Las 4 que se funden quedan en otra respuesta

02 · PREPARACIÓN, UNA SOLA VEZ

Antes del primer producto, tres llamadas

El seller se identifica, consulta qué puede hacer y dice a qué dirección quiere recibir los avisos.

1

Token

`POST /oauth/token`

15 minutos. Lo da el proveedor de identidad; no es parte de la API del PIM.

2

Quién soy

`GET /me`

Negocios, tiendas, alcances, ambiente, límites y uso del día.

3

Dónde recibo los avisos

`POST /webhooks`

URL + eventos. La respuesta trae, solo esta vez, el secreto para verificar la firma.

Estas tres llamadas no se repiten por producto. Solo al enchufarse la primera vez.

03 · PREDECIR LA CATEGORÍA

Un GET resuelve la primera pregunta: ¿qué es este producto?

La IA sugiere categorías con un número de confianza. La puerta sigue siendo un GET como el predictor de Mercado Libre.



confidence es un número de Liverpool. Mercado Libre no lo manda, solo ordena por probabilidad.

El atributo viaja como en Mercado Libre: `id` · `value_id` · `value_name`

categoryId = id de la plantilla · webCategory la asigna Liverpool, el tercero no la manda

04 · BAJAR LA PLANTILLA

Una sola llamada trae todo lo que necesita el seller

Atributos, tipos, valores cerrados, obligatorios, agrupación y versión. Ya no hay rutas aparte por lista o por marca.

```
GET /categories/CAT-DEMO-0412?business=MKP
```

Baterías externas

version 7 · domainId: POWER_BANKS

BRAND
Marca `list`
requerido
Marca Demo

CAPACIDAD
Capacidad `number + unit`
requerido
mAh

TIPO_CONECTOR
Tipo conector `list`
requerido · multi
USB-C · USB-A

COLOR
Color `list`
requerido · variación
Negro · Blanco

GTIN
GTIN `string`
por variante

MATERIAL
Material `list`
recomendado
Aluminio

BRAND, GTIN y los valores están en la misma respuesta. No hay /brands ni /value-lists.
El esquema JSON se queda del lado del integrador: Mercado Libre no lo da, el seller valida en casa.

05 · ANTES DE CREAR NADA

Dos ventanas de seguridad: validar y buscar

Validar es opcional (prueba hasta 100 productos sin crear). Buscar sí es obligatorio: si el producto ya existe, no se cataloga.

OPCIONAL · HASTA 100 PRODUCTOS

POST `/validations`

Mismo JSON del alta, sin crear nada. Por producto dice qué está mal y qué pasaría al mandarlo.

```
ERRORS[]
EAN_CHECK_DIGIT · GTIN de la variante BLA
El dígito verificador no corresponde a un EAN-13
VALUE_NOT_IN_LIST · TIPO_CONECTOR = "USB C"
No está en la lista; manda el id del valor (USB_C)
```

OBLIGATORIO · DECIDE EL CAMINO

GET `/products/search`

Por GTIN o por texto. Si existe, trae el SKU y si Mirakl ya lo acusó. Nunca dice qué seller es dueño.

```
DOS POSIBLES RESPUESTAS
results: [] → no existe, toca crear
results: [ {sku, allowedAction, offer} ]
→ ya existe, se mira allowedAction
```

Validar atrapa errores en frío · Buscar bifurca el flujo
Si search encuentra el producto, aquí termina el camino de catalogar.

06 · LA BIFURCACIÓN

La búsqueda responde con allowedAction. Solo uno cataloga.

Los otros cinco valores dicen qué sigue, pero no crean la ficha. Esto evita que el seller intente dar de alta algo que ya existe.

CATALOGA

CREATE_PROPOSAL

Crear propuesta

El único que cataloga. Entra al flujo 1 o al flujo 2.

A MIRAKL

CREATE_OFFER

Crear oferta

La ficha ya existe. La oferta se crea en Mirakl.

A MIRAKL

MULTI_OFFER

Multi-oferta

Varios sellers sobre la misma ficha. Mirakl.

A MIRAKL

STOCK_OUT_OFFER

Oferta sin stock

Oferta sobre un producto agotado. Mirakl.

NO CREA NADA

REQUEST_EAN_RELEASE

Pedir liberar EAN

No se manda la ficha.

NO CREA NADA

NOT_ALLOWED

No permitido

En Mercaderías, un producto que ya existe no se duplica.

Si aun así se manda el alta de un producto que ya existe, el cuerpo solo lleva catalogProductId y no reenvía la ficha.

07 · MANDAR UNO

POST /proposals · la respuesta vuelve en el momento

El cuerpo lleva atributos del producto, variantes con attribute_combinations y, si hay URL, las fotos.

PETICIÓN DEL SELLER

POST /proposals

```
{
  business: "MKP",
  externalRef: "DEMO-PB20",
  categoryId: "CAT-DEMO-0412",
  categoryVersion: 7,
  title: "PB-DEMO-20 Marca Demo",
  attributes: [...],
  variations: [
    // Negro, Blanco · con GTIN e imágenes
  ],
  logistics: { package: {...} }
}
```

instantáneo

RESPUESTA DEL PIM

201 Created

```
{
  proposalId: "PROP-000123",
  status: "IN_APPROVAL",
  correctionDueAt: null
}
```

Flujo 1 · un producto · respuesta en el momento · lo lento empieza después

Precio e inventario no entran por esta API — la oferta es de Mirakl · Idempotency-Key en el header para reintentos seguros

08 · MANDAR UN LOTE

POST /batches · hasta 1,000 productos de golpe

Acusa en el momento y procesa en segundo plano. Un producto malo no detiene a los demás.

1,000

productos máximo por llamada

LLAMADA

`POST` `/batches`

· al instante ·

`202` Accepted · procesando

```
batchId: "BATCH-000045" · received: 1000
```

después, un aviso

AVISO **batch.completed**

El PIM llama al webhook del seller cuando el lote termina. Trae el resumen.

LUEGO `GET` `/batches/{batchId}/items` resultado por producto, ligado a su externalRef

Flujo 2 · lote asíncrono · se procesa, se avisa, se consulta por partes

Un negocio por lote · Idempotency-Key por lote · lo que falla se reenvía con la misma referencia

09 · EL CICLO ASÍNCRONO · CORRECCIONES

Lo lento pasa en segundo plano. El seller no espera, le avisamos.

Si algo falta o está mal, llega un aviso con los motivos campo por campo. El seller corrige con PATCH y sigue.



AVISO `proposal.correction_required`

```
REASONS[]
attribute: "TITULO"
code: "SCORE_BELOW_THRESHOLD"
El título no dice qué es el producto;
incluye el tipo y la capacidad.
```

PATCH `/proposals/PROP-000123`

Con `If-Match: "4"`: corrige sobre la versión que leyó. Si alguien la cambió antes, responde 409.

```
{
  title: "Batería externa de 20,000 mAh
  con carga rápida USB-C"
}
```

El seller corrige solo lo que se pidió · campo por campo · versionado con If-Match

10 · EL CICLO ASÍNCRONO · SKU, IMÁGENES Y PUBLICACIÓN

SAP asigna el SKU · se completan imágenes · los canales publican

Entre canales la propuesta pasa por tres estados y, para cada uno, el PIM manda un aviso.

AVISO
product.sku_created
SKU y EAN por variante

LLAMADA
media/uploads + PUT /media
Si falta imagen (no hay URL pública)

AVISO
publication_sent
Enviado a cada canal

AVISO
product.published
El canal confirmó

VARIANTS[]

DEMO-PB20-NEG	SKU 1000000001
DEMO-PB20-BLA	SKU 1000000002

Cada variante trae también su EAN de 13 dígitos.

SI NO HABÍA URL PÚBLICA

POST /media/uploads
→ URL firmada por archivo (vence en 15 min)

PUT /proposals/.../media
→ fija el juego por variante (principal + orden)

CHANNELS[]

STORE	PUBLISHED
MIRAKL	PUBLISHED

Estados por canal: PENDING, SENT, PUBLISHED, FAILED, RETIRED.

SAP, imágenes y canales — todo asíncrono, todo con su aviso correspondiente.

11 · MODIFICACIONES POST-PUBLICACIÓN

Cambiar algo ya publicado: con permiso y en orden

Primero el seller pregunta qué puede cambiar; luego manda el cambio; después recibe el aviso.

1 ¿Qué puedo cambiar?

`GET` `/products/{id}/permissions`

TEXTOS — editable
Título · Descripción

ATRIBUTOS — requiresApproval
Capacidad · Material · Variantes nuevas

SKU · EAN · NEGOCIO — readOnly
No cambia nunca

2 Mandar el cambio

`POST` `/products/{id}/modifications`

Cambios en **atributos**, textos, imágenes o **variantes nuevas**. Con `If-Match`.

3 El aviso de cierre

AVISO `product.updated`

Trae el **origen del cambio** y la versión aplicada.

Ningún SKU, EAN o negocio se puede modificar · atributos piden aprobación · textos son libres.

12 · RED DE SEGURIDAD Y CIERRE

Si el webhook se cae, nada se pierde

Dos capas de respaldo: todos los avisos vienen firmados y existe un feed para releerlos en orden.

FEED DE RESPALDO

`GET` `/events?cursor=...`

Recupera en orden los avisos de los últimos 30 días. Si el webhook del seller cayó, se lee desde el cursor donde se quedó.

FIRMA HMAC

```
X-Liverpool-Signature: v1=<hmac>
X-Liverpool-Event-Id: EVT-000106
```

El seller verifica · descarta repetidos por `eventId` · ordena por `version`.

① **Solo 8 llamadas catalogan**

las otras 33 son el ciclo que viene después

② **Lo lento pasa en segundo plano**

SAP, canales, imágenes · todo tiene su aviso

③ **El seller nunca espera**

le avisamos con webhooks firmados y respaldados por un feed